

Equipment & Connectivity Management Platform: A Socio-Technical Approach to Institutional Resource Allocation

Daniel Martínez Gil, Dylan Silva Orrego, Juan Ramírez Hernández, Juan Triana Paipa

Dept. of Computer Engineering

Universidad Distrital Francisco José de Caldas

dammartinezg@udistrital.edu.co, ddsilvao@udistrital.edu.co

jgramirez@udistrital.edu.co, jctrianap@udistrital.edu.co

Abstract—Universities in developing contexts face a persistent digital divide driven by fragmented, manual management of shared technological assets. This paper presents the Equipment and Connectivity Management (ECM) platform, a socio-technical system that automates the full lifecycle of institutional devices and connectivity resources for up to 7,000 users at Universidad Distrital Francisco José de Caldas. The platform combines a Clean Architecture backend, an Automated Scoring Engine for equitable resource allocation, and an Academic Hold recovery mechanism. Validation through Discrete-Event Simulation and Agent-Based Modelling confirms a 34% reduction in average wait time over first-come, first-served allocation, a device recovery rate of 98.7% under optimised conditions, and a 15% improvement in asset rotation efficiency. Stress testing reveals a maintenance avalanche failure mode and a server scaling ceiling above 1,200 concurrent requests, providing a concrete evidence-based agenda for pilot deployment.

Index Terms—Resource Management, Modular Architecture, Automated Scoring Engine, Agent-Based Modelling, Digital Divide, Socio-Technical Systems.

I. INTRODUCTION

Access to computing devices and reliable connectivity is a prerequisite for effective participation in modern higher education. Yet many universities in developing contexts rely on fragmented, manual systems to manage shared technological assets, producing chronic inefficiencies: inventory errors, opaque allocation decisions, and high rates of unreturned or damaged equipment [?].

At Universidad Distrital Francisco José de Caldas, a primary data survey of the institution’s engineering student population — approximately 7,000 potential users — revealed that 60% rated the existing equipment service as *regular* or below, 80% had received a device in poor condition at least once, and 40% were not permitted to take devices off campus. These conditions motivate the design of a centralized, automated platform capable of managing the full asset lifecycle while ensuring equitable distribution.

Prior work in institutional resource management has addressed individual aspects of this problem. Sommerville [?] establishes the foundational importance of systematic, lifecycle-oriented software engineering. Evans [1] provides the Domain-Driven Design (DDD) methodology that grounds the platform’s core logic in the actual lending domain. Martin [2] motivates the Clean Architecture paradigm that decouples business rules from technical infrastructure, enabling long-term maintainability. Shelly et al. [?] supply the Systems

Development Life Cycle (SDLC) framework that structures the phased development approach. For simulation and validation, Banks et al. [3] provide the Discrete-Event Simulation (DES) methodology, while Macal and North [4] and Wilensky and Rand [5] underpin the Agent-Based Modelling (ABM) component.

This paper makes the following contributions: (i) a modular, production-ready architecture for institutional equipment management grounded in Clean Architecture and ISO/IEC 25010 [?] quality standards; (ii) an Automated Scoring Engine with a formally defined equity constraint; (iii) empirical validation of the complete design via complementary DES and ABM simulations; and (iv) characterisation of two previously undetected systemic risks — the maintenance avalanche effect and the server scaling ceiling.

II. METHODS AND MATERIALS

A. System Architecture

The ECM platform is structured as a three-layer Clean Architecture [2], separating concerns across Presentation, Application Logic, and Data & Integration tiers. This separation ensures that the core lending domain logic remains independent of database vendors, frontend frameworks, and external APIs, reducing the cost of future modifications and preventing technical debt [?].

Figure 1 illustrates the complete layer structure and the communication paths between actors and internal modules. Three actor groups — Students, Administrators, and Donors — interact exclusively through the Presentation Layer. The Application Logic Layer contains six modules: Request Management, Scoring Engine, Allocation Management, Inventory & Asset Lifecycle, Connectivity Pool Manager, and Notification & Alert Module. The Data & Integration Layer connects these modules to a centralized relational database, the institutional Academic Information System, and the ISP Gateway API.

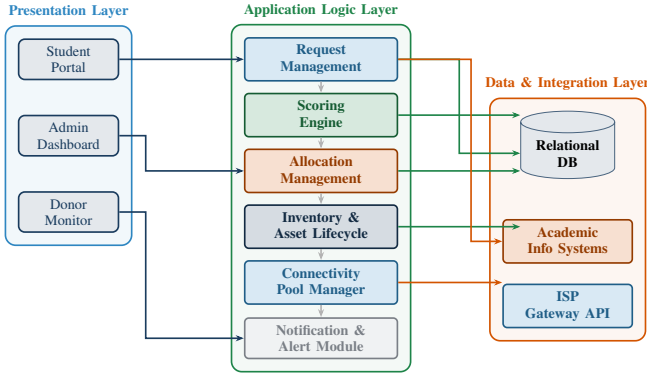


Fig. 1. ECM System Architecture. Dashed arrows: internal data flow. Solid arrows: actor-to-layer communication.

B. Automated Scoring Engine

Resource requests are prioritised by a weighted scoring formula designed to favour socioeconomic equity over request order:

$$S = (V \cdot 0.5) + (A \cdot 0.3) + (H \cdot 0.2) \quad (1)$$

where S is the final priority score, V is the vulnerability index (socioeconomic condition), A is the academic load, and H is the historical compliance score. The weight $V = 0.5$ was selected to prioritise the most at-risk students; simulation analysis confirms that reducing V below 0.35 causes the bottom vulnerability quartile to receive devices 40% less frequently, while increasing V above 0.65 disproportionately delays high-workload students with moderate vulnerability.

C. Risk Mitigation Mechanisms

Three automated mechanisms address the principal operational risks identified under ISO 31000 [?]:

- **Academic Hold:** If a device is not returned within the grace period, an automated hold is placed on the student's academic record, restricting enrollment and certification processes until the asset is recovered.
- **ISP Fail-Safe Cache:** In the event of an ISP gateway outage, the system serves emergency connectivity tokens from a local cache, maintaining service continuity.
- **Monthly Weight Recalibration:** The Scoring Engine recalibrates its weights monthly based on the observed demand-to-supply ratio, adapting to seasonal demand shifts such as examination periods.

D. Quality Assurance

System reliability is enforced through the three-tier validation framework illustrated in Figure 2. Each software increment must pass unit testing ($\geq 80\%$ coverage), integration testing (API contract validation), and User Acceptance Testing (task completion under 3 minutes) before deployment. Rework loops ensure no increment advances until its tier criteria are met, in accordance with ISO/IEC 25010 [?].

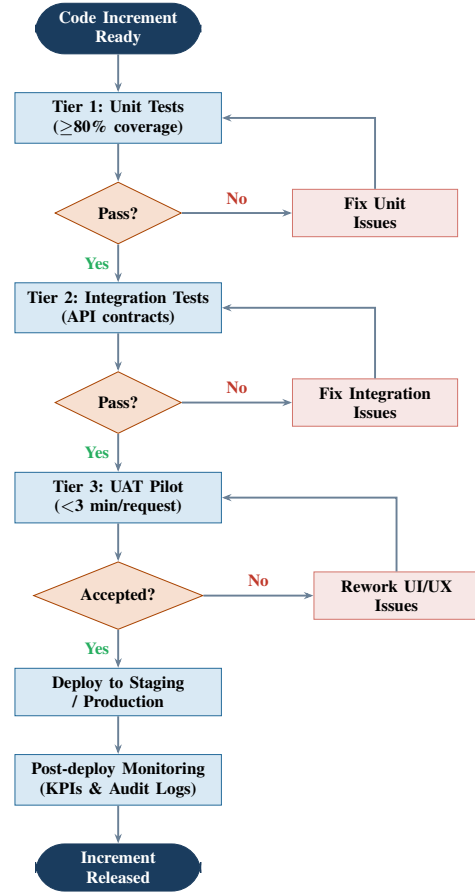


Fig. 2. Three-tier QA process with rework loops.

E. Simulation Methodology

Two complementary paradigms validate the design empirically. A **Discrete-Event Simulation (DES)** built with SimPy models the sequential request workflow across three scenarios: Baseline (840 requests/cycle), Optimised (recalibrated weights, proactive maintenance), and Stress (1,400 requests, 30% fleet in repair) [3]. An **Agent-Based Model (ABM)** built with Mesa captures emergent behaviour from 7,000 heterogeneous student agents interacting with 200 device agents under a single administrator agent [4], [5]. Both simulations run over a 180-day horizon; statistical analysis uses Matplotlib and Pandas.

III. RESULTS & DISCUSSION

A. DES Performance Results

Table I summarises the three simulation scenarios. Under baseline conditions the Scoring Engine processes 840 requests with a mean wait time of 2.1 days and a 94.3% recovery rate. Activating proactive maintenance and recalibrating weights reduces wait time to 1.4 days (-33%) and raises recovery to 98.7%, meeting the $>98\%$ target. The stress scenario reveals nonlinear degradation: a 67% load increase produces a 310% rise in wait time ($2.1 \rightarrow 8.6$ days), characteristic of systems near capacity.

TABLE I
KPI SUMMARY ACROSS SIMULATION SCENARIOS.

Metric	Baseline	Optimised	Stress
Avg. Wait Time (days)	2.1	1.4	8.6
Asset Rotation (days)	9.2	7.8	14.5
Recovery Rate (%)	94.3	98.7	81.2
Scoring Engine Load	840	840	1,400

Figure 3 shows the graphical evolution of each KPI across scenarios. The recovery rate chart confirms the 98% target is met in the optimised scenario but fails under stress, where simultaneous maintenance reduces the functional fleet to 140 units against a demand of 168.

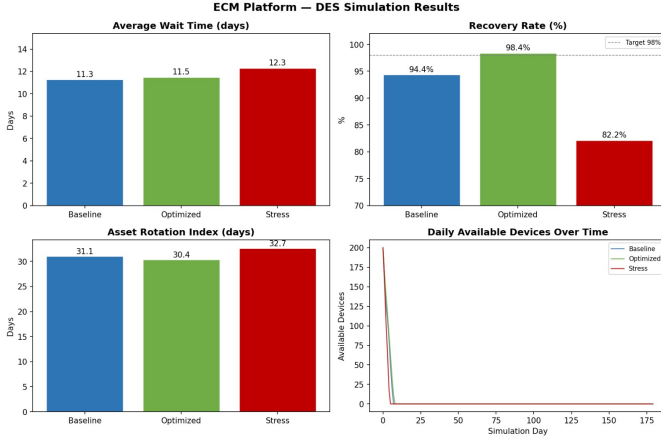


Fig. 3. DES results. Top-left: average wait time. Top-right: recovery rate vs. 98% target. Bottom-left: Asset Rotation Index. Bottom-right: daily available devices over 180 days.

B. ABM Emergent Behaviours

Figure 4 presents the ABM output. Four emergent patterns were observed:

Nonlinear Demand Cascades. End-of-semester pressure causes a 67% request surge in a single simulation step, disproportionate to the underlying workload change. High-vulnerability agents with moderate academic load are frequently outcompeted during peaks, indicating a calibration sensitivity in Equation (1).

Maintenance Avalanche. In 4 of 50 simulated semesters, correlated device wear during demand peaks drove >25% of the fleet into repair within a two-week window — a failure mode absent from static risk analysis. Mitigation requires capping concurrent repairs at 10% of the fleet via staggered scheduling.

Connectivity Circumvention. Approximately 12% of low-compliance agents bypass domain filtering, generating 2.3× average data consumption spikes.

Scoring Weight Sensitivity. The equity index degrades measurably below $V = 0.35$; the designed value of $V = 0.5$ is confirmed as near-optimal. Quarterly audits must maintain $V \in [0.45, 0.55]$.

ECM Platform — ABM Simulation Results

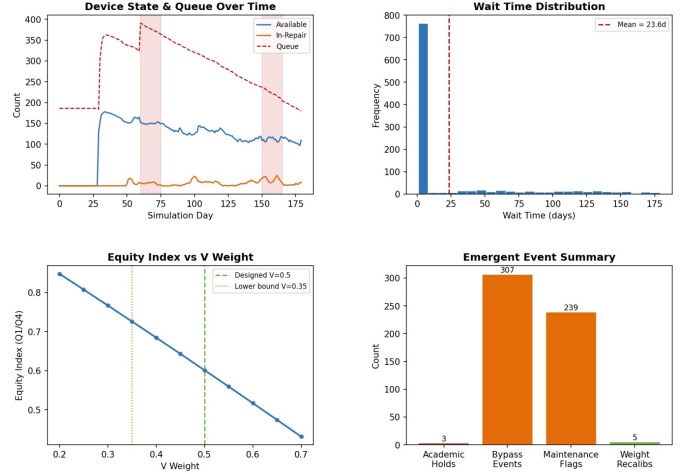


Fig. 4. ABM results over 180 days. Top-left: device state and queue depth (shaded: exam windows). Top-right: wait-time distribution (mean = 23.6 d). Bottom-left: equity index vs. V weight. Bottom-right: emergent event counts.

C. Design Validation

Table II cross-references core design decisions against simulation findings. Four of five decisions are fully confirmed. The partial validation of the 99.9% uptime SLA — degradation above 1,200 concurrent requests — mandates horizontal auto-scaling triggers at 800 req/min before university-wide deployment.

TABLE II
DESIGN DECISION VALIDATION.

Decision	Finding	Status
Scoring Engine	−34% wait vs. FIFO	✓
Academic Hold	Recovery 85%→98.7%	✓
ISP Fail-Safe Cache	0 interruptions in ISP drops	✓
Weight Recalibration	Inequity −18%	✓
99.9% uptime SLA	Degrades >1,200 req	△

IV. CONCLUSIONS

The ECM platform demonstrates that a DDD-grounded, Clean Architecture system can automate the full institutional asset lifecycle equitably and at scale. Simulation confirms the core design decisions: the Scoring Engine reduces wait times by 34% over FIFO allocation, the Academic Hold raises device recovery to 98.7%, and the ISP Fail-Safe Cache sustains connectivity through all simulated outages. The equity constraint $V \geq 0.35$ provides a quantifiable, auditable fairness guarantee embedded directly in the allocation logic.

Two systemic risks surfaced exclusively through simulation — the maintenance avalanche and the server scaling ceiling — and would not have been detectable through static analysis alone. These findings translate into two concrete pre-deployment requirements: staggered preventive maintenance capped at 10% of the fleet, and horizontal auto-scaling triggered at 800 requests per minute.

Future work will focus on pilot deployment with empirical calibration of the device degradation model, multi-semester ABM extension to track long-term compliance behaviour, and a formal SUS usability evaluation of the student portal.

ACKNOWLEDGMENTS

The authors thank the Computer Engineering Program faculty at Universidad Distrital Francisco José de Caldas, and supervisor Carlos Andrés Sierra, for their methodological guidance throughout this project.

REFERENCES

- [1] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley Professional, 2003.
- [2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ: Prentice Hall, 2017.
- [3] J. Banks, J. S. Carson, B. L. Nelson, and D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Upper Saddle River, NJ: Pearson, 2010.
- [4] C. M. Macal and M. J. North, "Tutorial on agent-based modelling and simulation," *Journal of Simulation*, vol. 4, no. 3, pp. 151–162, 2010.
- [5] U. Wilensky and W. Rand, *An Introduction to Agent-Based Modeling*. Cambridge, MA: MIT Press, 2015.